RPG IV Mise en œuvre

Gestion des erreurs

Généralités (1/2)

- Qu’est-ce qu’une erreur ?
  - Un événement (une exception) qui provoque l’arrêt en anomalie du programme
- Qu’est-ce que traiter une erreur ?
  - Intercepter l’événement afin d’éviter l’arrêt du programme
  - Effectuer un traitement pour continuer l’exécution
- Une erreur est-elle vraiment une erreur ?
  - Pas toujours : accéder à un enregistrement pour tester sa non-existence

Généralités (2/2)

- RPG IV permet de gérer les erreurs de plusieurs manières
  - Instruction par instruction

    Par l'extension (E) des codes opérations qui le supportent, ou les indicateurs d'erreurs en format fixe
  - Par bloc d'instructions

    Par des blocs MONITOR
  - Par programme

    Par des sous-routines particulières : *PSSR et INFSR

Extension & indicateur

- Pour les codes opérations qui le supportent
  - Indicateur en position <<
    - Erreur si l’indicateur est à *ON après l’instruction
    - Format fixe uniquement
  - Extension (E) sur le code opération
    - Erreur si la BIF %error renvoie *ON après l’instruction
    - Format libre et fixe

Exemple – Extension & indicateur

```
 * Le fichier client contient les zones :
 *  NUMCLI INTEGER        => Clé
 * ...
 *  CPLCLI CHARACTER (5)
FCLIENTS   IF   E           K DISK    RENAME( CLIENTS : FMTCLI )

 * Lecture du client n° 10
C                   EVAL      NUMCLI = 10
C     NUMCLI        CHAIN     CLIENTS                          0102
C                   IF        *IN02
 * Anomalie sur le fichier
C                   ENDIF
C                   IF        *IN01
 * Enregistrement trouvé
C                   ENDIF

C     NUMCLI        CHAIN(E)  CLIENTS
C                   IF        %error
 * Anomalie sur le fichier
C                   ENDIF

C     NUMCLI        CHAIN(E)  CLIENTS
C                   EVAL      CPLCLI = '69009'

 /free
  CHAIN(E) NUMCLI CLIENTS ;
  IF %error ;
     // anomalie ...
  ENDIF;
```

Avec indicateur

Avec (E) et %error

A ne pas faire : erreur silencieuse

Syntaxe format libre

Bloc monitor (1/2)

- Un bloc d'instructions monitorées
  - Commence par MONITOR
  - Se termine par ENDMON
- Les erreurs sont interceptées
  - Individuellement

    ON-ERROR code erreur
  - Par groupe

    ON-ERROR erreur1:erreur2
  - Par mot-clé

ON-ERROR *PROGRAM	Erreurs programme
ON-ERROR *FILE	Erreurs fichier
ON-ERROR *ALL	Toutes les erreurs
ON-ERROR	= *ALL

Bloc monitor (2/2)

- L'ensemble des erreurs est dans le fichier QRNXMSG
  - DSPMSGD RANGE(*FIRST *LAST) MSGF(QRNXMSG) DETAIL(*BASIC)

    Remplacer les lettres par des 0

    RNQ0102 → 000102 : division par 0
- En cas d'erreur
  - Les clauses ON-ERROR sont parcourues séquentiellement jusqu'à une clause correspondante
  - Si plusieurs clauses correspondent, c'est la première qui est exécutée
  - Il faut donc organiser les clauses de la plus particulière à la plus générale

Exemple – MONITOR (1/2)

```
     * Le fichier client contient les zones :
     *  NUMCLI INTEGER        => Clé
...
     *  CPLCLI CHARACTER (5)
     FCLIENTS   IF   E           K DISK    RENAME( CLIENTS : FMTCLI )

     C                   MONITOR
     C                   READ      CLIENTS
     C                   IF        not %eof
     C                   EVAL      %subst( CPLCLI : 1 : 5 ) = 'X'
     C                   ENDIF
     C                   ON-ERROR  1211
     * Traiter fichier non ouvert
     C                   ON-ERROR  *FILE
     * Traiter les autres erreurs sur fichier
     C                   ON-ERROR  00100:00121
     * Traiter les erreurs sur tableau et chaine de caractères
     C                   ON-ERROR
     * Toutes les autres erreurs
     C                   ENDMON
```

Début du bloc monitoré

Interception des erreurs

Exemple – MONITOR (2/2)

```
     * Le fichier client contient les zones :
     *  NUMCLI INTEGER        => Clé
...
     *  CPLCLI CHARACTER (5)
    FCLIENTS   IF   E           K DISK    RENAME( CLIENTS : FMTCLI )


     /free
      MONITOR ;
         READ CLIENTS ;
         IF not %eof ;
            %subst( CPLCLI : 1 : 5 ) = 'X' ;
         ENDIF;
      ON-ERROR  1211 ;
      // Traiter fichier non ouvert
      ON-ERROR  *FILE ;
      // Traiter les autres erreurs sur fichier
      ON-ERROR  00100:00121 ;
      // Traiter les erreurs sur tableau et chaine de caractères
      ON-ERROR ;
      // Toutes les autres erreurs
      ENDMON;
```

Début du bloc monitoré

Interception des erreurs

Sous-routines

- Des sous-routines particulières permettent de traiter les erreurs
  - Spécifiques à un fichier

    Sous-routine indiquée par le mot-clé INFSR sur la déclaration du fichier
  - Générale au programme

    Pour toutes les erreurs non interceptées : *PSSR

    Si une erreur intervient dans *PSSR, la sous-routine est appelée à nouveau

Exemple – *PSSR

```
FCLIENTS   IF   E           K DISK    RENAME( CLIENTS : FMTCLI )
D Erreur          S               N   INZ( *OFF )
C                   READ      CLIENTS
C                   IF        not %eof
C                   EVAL      %subst( CPLCLI : 1 : 5 ) = 'X'
C                   ENDIF
C                   RETURN
 * Sous-routine de gestion des erreurs programme
C     *PSSR         BEGSR
 * Ne pas boucler sur les erreurs de *PSSR lui-même
C                   IF        Erreur
C                   EVAL      *INLR = *ON
C                   RETURN
C                   ENDIF
 * Traiter l'erreur et sortir proprement
C                   EVAL      Erreur = *ON
C     '*PSSR - err' DSPLY
 * Traiter l'erreur ici
C                   ENDSR
 /free
  BEGSR *PSSR ; // Sous-routine de gestion des erreurs programme
  MONITOR ;     // Ne pas boucler sur les erreurs de *PSSR lui-même
     // Traiter l'erreur
  ON-ERROR ;
     DSPLY '*PSSR – Sortie forcée'
  ENDMON ;
  // Sortir
  *INLR = *ON ;
```

Evite les boucles infinies en cas d'erreur dans *PSSR

Exemple – INFSR

```
 * Le fichier client contient les zones :
 *  NUMCLI INTEGER        => Clé
 * ...
 *  CPLCLI CHARACTER (5)
FCLIENTS   IF   E           K DISK    RENAME( CLIENTS : FMTCLI )
F                                     INFSR( ERRCLIENTS )

FARTICLE   IF   E           K DISK    INFSR( *PSSR )


C                   READ      CLIENTS
C                   IF        not %eof
C                   EVAL      %subst( CPLCLI : 1 : 5 ) = 'X'
C                   ENDIF

C                   RETURN

C     ERRCLIENTS    BEGSR
 * Traiter l'erreur
C                   ENDSR

C     *PSSR         BEGSR
 * Sortir proprement
C                   ENDSR
```

Sous-routine spécifique au fichier CLIENTS

*PSSR peut être utilisée avec INFSR

Complément

- Pour les erreurs programme et fichier
  - %status retourne le statut le plus récent